

Лабораторна робота N6

Серіалізація. Лямбда-вирази та Stream API.

Задача. Навчитись створювати запити з використанням лямбда-виразів та Stream API. Навчитись використовувати серіалізацію/десеріалізацію.

Завдання.

1. Необхідно реалізувати щонайменше три запити для підрахунку мікрооб'єктів. Наприклад: а) кількість активних мікрооб'єктів; б) кількість мікрооб'єктів із рівнем життя понад 50%; в) кількість мікрооб'єктів у правій половині екрана тощо.
2. У вікні програми відображається лише певна частина ігрового світу, яка не повинна перевищувати 25% його загального розміру.
3. Реалізувати можливість змінювати частину ігрового світу, яку спостерігає користувач (будь яким способом)
4. Повинна бути реалізована мінікарта, яка дозволяє прискорену навігацію: 1) На мінікарті повинна бути позначена видима область універсального об'єкта. 2) Рух видимої області універсального об'єкта відповідно змінює мінікарту. 3) Макрооб'єкти та мікрооб'єкти позначені на мінікарті. 4) Рух та взаємодія мікрооб'єктів призводить до відповідних змін на мінікарті.
5) Натискування лівої кнопки миші на мінікарті призводить до переміщення видимої області універсального об'єкта.
5. Повинна бути запрограмована серіалізація/де-серіалізація всіх об'єктів у текстовий/бінарний/XML файл. Серіалізація/де-серіалізація обов'язково повинна зберігати не тільки власне інформацію про стан макро- та мікро-об'єктів, але й про їх позицію на екрані.
6. При серіалізації/де-серіалізації обов'язково повинні використовуватись діалогові вікна, щоб запитати у користувача ім'я файлу та його розташування на диску (наприклад можна використовувати системне діалогове вікно FileChooser).
7. Продемонструвати використання лямбда-виразу.
8. Продемонструвати використання операція з потоком Stream.

Контрольні питання

1. Дайте визначення лямбда-виразу в Java. Опишіть його синтаксис та базові сценарії застосування.
2. Опишіть процес проектування та використання об'єктів Comparator за допомогою лямбда-виразів. Наведіть приклад сортування колекції.

3. Що таке функціональний інтерфейс?
4. Яку роль відіграє анотація `@FunctionalInterface`?
5. Яким чином відбувається виклик (виконання) коду, описаного в тілі лямбда-виразу? Поясніть зв'язок з абстрактним методом інтерфейсу.
6. У чому полягає відмінність між лямбда-виразом та анонімним внутрішнім класом з погляду компіляції та створення об'єктів у пам'яті (Heap)?
7. Які обмеження накладаються на використання зовнішніх локальних змінних (змінних із навколишнього контексту) всередині тіла лямбда-виразу? Що означає термін `effectively final`?
8. Назвіть основні типи вбудованих функціональних інтерфейсів у пакеті `java.util.function` (наприклад, `Predicate`, `Function`, `Consumer`, `Supplier`). Опишіть призначення кожного з них.
9. Що таке посилання на методи (Method References, оператор `::`)?
10. Що таке проміжна операція з потоком `Stream`?
11. Що таке термінальна операція з потоком `Stream`?
12. Як створити потік `Stream`?
13. Як відфільтрувати елементи потоку `Stream` за певним принципом?
14. Як порахувати елементи потоку `Stream`?
15. Як використовувати операції `noneMatch`, `allMatch`, `anyMatch` над потоком `Stream`?
16. Як використовувати операції `map` та `flatMap` над потоком `Stream`?
17. Які три стадії проходить життєвий цикл потоку `Stream`?
18. Що таке інтерфейс `java.io.Serializable`? Які методи він містить і яка його головна роль для JVM?
19. Які класи стандартної бібліотеки Java використовуються для безпосереднього запису об'єкта у файл (серіалізації) та його зчитування (десеріалізації)?
20. Яким чином серіалізуються об'єкти, що містять посилання на інші об'єкти (так званий граф об'єктів)? Що станеться, якщо один із внутрішніх об'єктів не реалізує `Serializable`?
21. Поясніть призначення ключового слова `transient`.